

Lab 11: Connecting the Frontend to the Backend

Course: CST150: Web technologies

Objective

By the end of this lab, students will:

- Understand how the frontend communicates with the backend.
- Learn how to fetch and send data using APIs.
- Implement a simple frontend-backend connection using JavaScript and a backend server.

As always, don't just follow the workshop verbatim, tweak the changes and see the difference. Share your findings with the group.

Pre-requisites

- Completion of Lab 10 (Building REST APIs with Flask).
- Basic knowledge of HTML, CSS, JavaScript, and a backend language (Python/Node.js).
- Local server or online API for testing.
- Installed required libraries: Axios, Flask-CORS.

```
pip install flask-cors  
npm install axios
```

Lab Tasks

Task 1: Setting Up Flask Backend with CORS

1. Duplicate `CST150_lab10` into `CST150_lab11` folder.
2. Modify `app.py` to allow frontend requests:

```
from flask import Flask, jsonify, request
from flask_mysql import MySQL
from flask_cors import CORS

app = Flask(__name__)
CORS(app) # Enable CORS

app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = 'yourpassword'
app.config['MYSQL_DB'] = 'flask_db'

mysql = MySQL(app)

@app.route('/users', methods=['GET'])
def get_users():
    cursor = mysql.connection.cursor()
    cursor.execute("SELECT * FROM users")
    users = cursor.fetchall()
    cursor.close()
    return jsonify(users)

if __name__ == '__main__':
    app.run(debug=True)
```

Task 2: Creating the Frontend Interface

1. Create a new `index.html` file with the following structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Frontend-Backend Integration</title>
  <script defer src="script.js"></script>
</head>
<body>
  <h1>User List</h1>
  <ul id="user-list"></ul>
</body>
</html>
```

2. Create a `script.js` file to fetch data from the backend:

```
document.addEventListener("DOMContentLoaded", () => {
  fetch("http://127.0.0.1:5000/users")
    .then(response => response.json())
    .then(data => {
      const userList =
document.getElementById("user-list");
      data.forEach(user => {
        let li = document.createElement("li");
        li.textContent = `${user[1]} - ${user[2]}`;
        userList.appendChild(li);
      });
    })
    .catch(error => console.error("Error fetching users:",
error));
});
```

3. Open `index.html` in a browser and check if user data loads
-

Task 3: Sending Data from the Frontend to the Backend

1. Update `index.html` to include a user form:

```
<form id="user-form">
  <input type="text" id="name" placeholder="Name" required>
  <input type="email" id="email" placeholder="Email" required>
  <button type="submit">Add User</button>
</form>
```

2. Modify `script.js` to handle form submission:

```
document.getElementById("user-form").addEventListener("submit",
(e) => {
  e.preventDefault();
  const name = document.getElementById("name").value;
  const email = document.getElementById("email").value;

  fetch("http://127.0.0.1:5000/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, email })
  })
  .then(response => response.json())
  .then(data => {
    alert(data.message);
    location.reload();
  })
  .catch(error => console.error("Error adding user:", error));
});
```

3. Update `app.py` to handle POST requests:

```
@app.route('/users', methods=['POST'])
def add_user():
    data = request.get_json()
    cursor = mysql.connection.cursor()
    cursor.execute("INSERT INTO users (name, email) VALUES (%s, %s)", (data['name'], data['email']))
    mysql.connection.commit()
    cursor.close()
    return jsonify({'message': 'User added successfully'}), 201
```

4. Restart Flask and test adding users from the form.
-

Discussion Questions

1. Why do we use CORS when integrating frontend and backend?
 2. What are the advantages of using Fetch API vs Axios?
 3. How can we handle errors gracefully in API calls?
 4. What security measures should be added before deploying this system?
-

Bonus Challenge

- Implement an edit feature to update user details.
 - Implement a delete feature to remove users.
 - Style the UI using CSS for a better user experience.
-

Conclusion

In this lab, students successfully connected a frontend to a Flask backend, retrieved and displayed data, and sent new user data to the database. The next lesson will focus on advanced database integration using SQL features.

End of Lab 11